

Project Documentation & AI Usage Report

LLD Practice Platform Ø=ÜÐ

> A focused, production-grade prototype that helps software engineers practice Low-Level Design (LLD), submit solutions across structured dimensions, and receive explainable, evidence-based rubric feedback with iterative attempt tracking.

Built for the 2-Day Engineering Assignment & Candidate Evaluation.

Ø=Þ€ Quick Start (Running Locally)

Prerequisites

- **Node.js** v18+ (tested on Node v22.19.0)
- **npm** v9+ (tested on npm v10.9.3)

1. Run Backend Service

```
`bash
cd backend
npm install
npm run dev
`
```

Backend starts on: <http://localhost:4000> (API & Health Check at <http://localhost:4000/health>)

2. Run Frontend Client

In a new terminal:

```
`bash
cd frontend
npm install
npm run dev
`
```

Frontend runs on: <http://localhost:3000> (automatically proxies /api calls to the backend).

3. Run Automated Tests

```
`bash
cd backend
npm test
`
```

Runs 5 comprehensive automated test suites verifying domain invariants, state machines, deterministic checks, and end-to-end iteration tracking.

Ø<β The Core Learner Journey

\$\$\text{Select Problem} \rightarrow \text{Think \& Design} \rightarrow \text{Submit} \rightarrow \text{Rubric Feedback} \rightarrow \text{Compare History} \rightarrow \text{Refactor (Attempt 2)}\$\$

1. **Problem Selection**: Choose from classic LLD problems (**Multi-Level Parking Lot**, **Elevator Control System**, **Automated Vending Machine**).

2. **Design Studio**: Express architectural decisions across 4 structured dimensions:
 - **Scope & Assumptions**: Functional & non-functional constraints (concurrency, capacity, boundaries).
 - **Entities & Interfaces**: High cohesion classes, interface contracts, and method signatures.
 - **Design Patterns & Trade-offs**: Explicit pattern justifications (e.g. Strategy for pricing, State for elevator/machine).
 - **Class Diagram / Code**: Mermaid class relationships and pseudocode skeletons.
3. **One-Click Demonstrations**:
 - **Load Starter Scaffolding**: Scaffolds the problem structure for quick drafting.
 - **Load Sample Architecture**: Loads a canonical, multi-pattern architecture for instant evaluation testing.
4. **Explainable Evaluation Report**:
 - Rubric schema: `criterion -> score (1-5) -> evidence -> concern -> suggestion -> confidence`.
 - Cites actual evidence quotes from the candidate's submission.
5. **Attempt Progression Tracking**:
 - History drawer compares Attempt 1 vs Attempt 2 with score deltas (`+36%`) to reinforce iterative improvement.

Ø<BÛp System Architecture & Domain Model

The platform is designed following Domain-Driven Design (DDD) and Clean Architecture:

```

backend/
% % % src/
%   % % % domain/           # Enterprise Business Rules & Models
%   %   % % % enums/       # Difficulty, RubricDimension, SubmissionStatus, etc.
%   %   % % % models/      # Problem, Attempt, Submission, Evaluation, Rubric, FeedbackReport
%   %   % % % repositories/ # IProblemRepository, IAttemptRepository, ISubmissionRepository
%   %   % % % services/    # IEvaluator strategy interface
%   % % % % application/   # Application Business Rules (Use Cases)
%   %   % % % usecases/    # StartAttempt, SubmitSolution, GetHistory, ListProblems
%   % % % % infrastructure/ # Frameworks, Evaluators & Repositories
%   %   % % % evaluators/  # DeterministicRuleEvaluator, SemanticRuleEvaluator, GeminiLlmEvaluator,
CompositeEvaluator
%   %   % % % repositories/ # InMemoryProblemRepository, InMemoryAttemptRepository, etc.
%   %   % % % seed/        # Rich seed problem catalog with starter templates & sample architectures
%   % % % % presentation/  # Interface Adapters (Express Controllers & Routes)
%   %   % % % controllers/
%   %   % % % routes/
% % % % tests/             # Automated Unit & Integration Tests (Vitest)

```

Ø>Ýê Evaluation Architecture & Extensibility

```

`mermaid
flowchart LR
Sub[Submission Received] --> Store[(Persist SUBMITTED)]
Store --> CompEval[CompositeEvaluator]

subgraph Pipeline [Evaluation Pipeline]
CompEval --> Det[DeterministicRuleEvaluator]

```

```

Det -->|Flags Missing Entities / God Objects| Synth[Synthesize Feedback]
Det -->|Passes Gate| Deep[Intelligent Rubric Evaluator]
Deep -->|If GEMINI_API_KEY set| LLM[GeminiLlmEvaluator]
Deep -->|Offline / Default| Heuristic[SemanticRuleEvaluator]
end

```

```

Pipeline --> Merge[Aggregate & Synthesize FeedbackReport]
Merge --> Complete[(Persist COMPLETED & Update Submission)]
`

```

Pluggable Evaluators

1. `DeterministicRuleEvaluator`: Validates schema compliance, entity counts, interface declarations, missing core domain entities, and God Object anti-patterns.
2. `SemanticRuleEvaluator` (Offline Heuristic Engine): High-precision semantic analyzer assessing coupling, cohesion, design patterns, and concurrency safeguards without external API dependencies.
3. `GeminiLlmEvaluator`: Optional zero-shot LLM evaluator. Automatically activates when `GEMINI_API_KEY` is provided in `.env`, falling back gracefully to the heuristic engine if disconnected.
4. `CompositeEvaluator`: Unifies deterministic rules and intelligent rubric feedback into an explainable report.

Ø=Ü; Candidate Helping Guide Answers

Change Test A

> Today the learner submits text. Later the platform supports a class diagram. How much of your domain model changes?

- `Answer`: 0 changes to `Attempt`, `Problem`, `Evaluation`, or the practice loop.
- `Implementation`: Submissions accept an abstract `ISubmissionPayload`. Currently implemented as `StructuredTextPayload` and `DiagramPayload`. Adding interactive canvas or code execution merely requires adding a new payload implementation implementing `toTextSummary()`.

Change Test B

> Today feedback comes from one evaluator. Later you add a rule-based evaluator or human review. Can you add it without rewriting the practice flow?

- `Answer`: Yes, without modifying a single line in the practice flow or use cases.
- `Implementation`: The practice use case (`SubmitSolutionUseCase`) depends solely on the abstract `IEvaluator` interface via Dependency Injection. Any evaluator (e.g. `HumanReviewEvaluator`, `LintEvaluator`) can be plugged into `CompositeEvaluator` without touching the domain layer.

Practical Scale & State Handling

1. `Submission-First Persistence`: Submissions are saved immediately with status `SUBMITTED` *before* the evaluator executes. If an evaluator fails or times out, the user's work is never lost.
2. `Idempotency Guard`: Submissions enforce idempotency keys. Retrying a submission returns the existing evaluation without duplicate processing.
3. `Async Worker Ready`: At scale, the evaluation pipeline is decoupled via a job queue (e.g. BullMQ / Redis) where the API acknowledges `SUBMITTED` and workers transition the state to `COMPLETED`.

Ø=ÜÁ Key Documentation Deliverables

- `[**docs/RESEARCH.md**](file:///e:/Cipher%20School/docs/RESEARCH.md)`: 1–2 page research note detailing the learner problem, existing tools (LeetCode, GitHub, Educative), key market gaps, and product direction.
- `[**docs/DESIGN.md**](file:///e:/Cipher%20School/docs/DESIGN.md)`: Detailed architectural design note, domain model, trade-offs, and Change Test answers.
- `[**docs/AI_USAGE.md**](file:///e:/Cipher%20School/docs/AI_USAGE.md)`: Detailed log of 5 meaningful AI-assisted decisions (what was suggested, accepted/rejected, and engineering rationale).

- `[**backend/tests/domain_and_evaluation.test.ts**](file:///e:/Cipher%20School/backend/tests/domain_and_evaluation.test.ts)`: Vitest test suite testing state transitions, idempotency, evaluators, and iteration progression.

AI Usage Log: Key AI-Assisted Decisions

Project: Low-Level Design (LLD) Practice Platform

Author: Aryaman Malik

Scope: Transparent log of meaningful AI-assisted suggestions, design choices, accepted/rejected ideas, and engineering rationale.

Decision 1: Submission Input Model — Free-form Text vs. Structured Canvas

- **AI Suggestion**:

The AI initially suggested supporting an open-ended Markdown editor where candidates write freely, arguing this offers the least friction and lets users copy-paste existing design notes.

- **Accepted / Rejected**:

Rejected the open-ended text box; Accepted a structured multi-component submission model.

- **Rationale**:

In LLD, unstructured text frequently leads to candidates skipping critical architectural aspects (e.g. omitting interface contracts, concurrency assumptions, or pattern trade-offs). An unconstrained text dump also forces the evaluation engine into fuzzy prompt parsing, resulting in inconsistent feedback. We selected a structured canvas with explicit sections: (1) Requirements & Scope Assumptions, (2) Class & Interface Contracts, (3) Design Patterns & Trade-offs, and (4) Structural Diagram / Code Skeleton. This ensures evidence is cleanly isolated for each rubric dimension while maintaining a low friction authoring experience.

Decision 2: Feedback Generation Strategy — Monolithic LLM Prompt vs. Multi-Stage Composite Pipeline

- **AI Suggestion**:

The AI suggested a direct prompt: "Act as a Principal Engineer and evaluate this submission from 1 to 100, providing feedback on SOLID principles."

- **Accepted / Rejected**:

Rejected the monolithic 1–100 score prompt; Accepted a Composite Evaluator with a Deterministic Gatekeeper + Criterion-by-Criterion Evidence Model.

- **Rationale**:

A generic 0–100 score lacks explainability and produces high variance between runs. Furthermore, if the learner omits core domain entities, burning LLM tokens to evaluate SRP on nonexistent classes is wasteful. Instead, we designed a `DeterministicRuleEvaluator` that verifies baseline structural requirements first (missing entities, God-class thresholds), followed by an intelligent evaluation mapped to a fixed rubric schema (criterion -> score (1-5) -> evidence -> concern -> suggestion -> confidence). This makes feedback reproducible and anchored in what the learner actually wrote.

Decision 3: Submission Lifecycle & Resilience — Synchronous In-Flight Evaluation vs. Persist-First State Machine

- **AI Suggestion**:

The AI suggested a simple request-response API: `POST /api/submissions` immediately runs the evaluation in memory and returns the evaluated result directly in the HTTP response.

- **Accepted / Rejected**:

Rejected synchronous in-memory evaluation; Accepted a Persist-First State Machine (`SUBMITTED` to \$

EVALUATING \$to\$ COMPLETED/FAILED) with Idempotency Guard.

- **Rationale**:

AI evaluations and in-depth semantic parsers introduce latency and external network dependencies. If an evaluation times out or encounters network degradation during an in-flight HTTP request, the candidate's entire solution would be lost. By persisting the submission immediately with status SUBMITTED and an idempotency key, we guarantee zero data loss. The evaluation runs asynchronously (or synchronously with safe fallback), and failed evaluations can be retried without re-authoring the solution.

Decision 4: Fallback & Offline Independence — Pure Cloud LLM vs. Dual-Mode Heuristic Engine

- **AI Suggestion**:

The AI suggested relying exclusively on the Google Gemini API or OpenAI API, requiring an API key for every run.

- **Accepted / Rejected**:

Rejected hard external API dependency; Accepted a Pluggable Strategy with an Offline Heuristic Engine.

- **Rationale**:

Evaluators and reviewers running this prototype in an interview or automated CI setting may not have active external API keys configured, or could experience quota limits. To ensure 100% testability and reliability, we implemented a SemanticRuleEvaluator heuristic engine that evaluates OOP coupling, inheritance, and pattern presence offline. When GEMINI_API_KEY is present, the system automatically uses GeminiLlmEvaluator; when absent or on failure, it seamlessly falls back to the heuristic engine without throwing runtime errors.

Decision 5: Attempt History & Practice Progression — Flat Submission List vs. Multi-Iteration Attempt Aggregate

- **AI Suggestion**:

The AI proposed a flat submissions table where each submit is simply another row associated with the problem.

- **Accepted / Rejected**:

Rejected flat submissions; Accepted an Attempt aggregate root encapsulating multiple Submission iterations.

- **Rationale**:

The primary learner problem identified in our research is the lack of an iterative learning loop (Try \$to\$ Feedback \$to\$ Refactor \$to\$ Compare). By establishing Attempt as an explicit aggregate with sequential iterations, the platform can compare Attempt 1 vs. Attempt 2 directly, showing learners the delta in their rubric scores (e.g. +2 on Extensibility after introducing the Strategy pattern) and reinforcing positive learning habits.